

Test 2

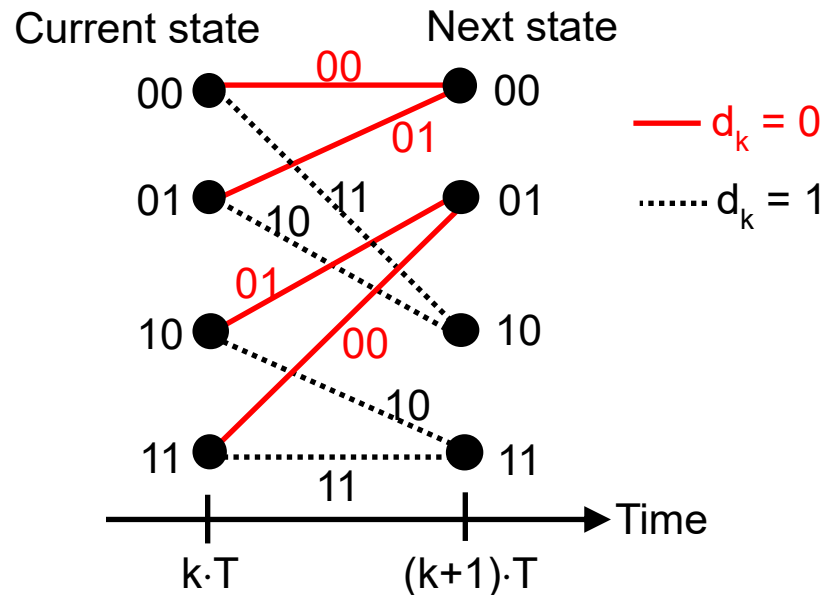
Coding Theory

Solutions

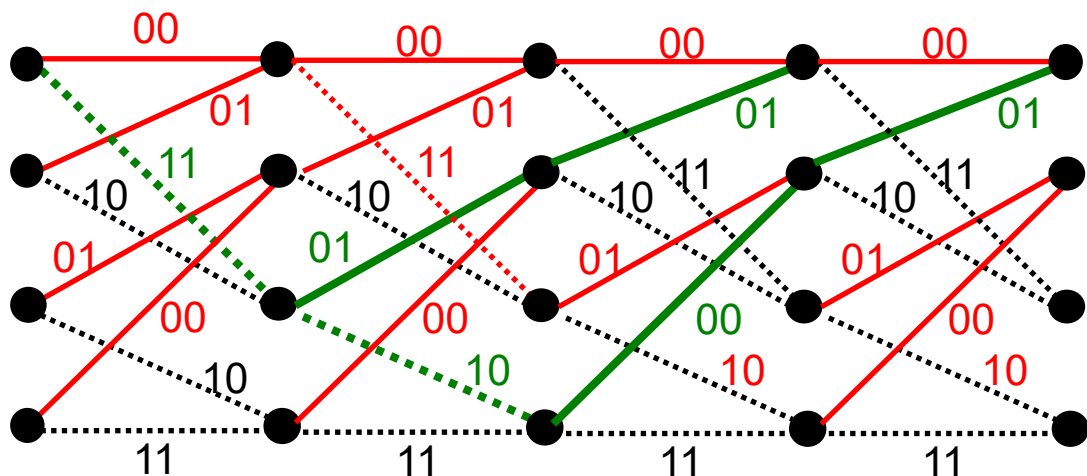
S Le Goff

Question 1

(a)



(b) We must search for the path with the smallest Hamming weight that leaves the state 00 and goes back to it.



The free distance is its Hamming weight. We find two such paths (shown above in green) and they have a Hamming weight of 4. Thus, we have $d_{free} = 4$.

(c) We can use the first term of the union bound expression to answer this question:

$$P_{eb} \sim \frac{N_{d_{free}}}{2} \cdot \text{erfc} \left(\sqrt{d_{free} R_c \frac{E_b}{N_0}} \right).$$

The coding rate R_c is equal to $1/2$ and the free distance d_{free} is equal to 4. We still need to determine the value of the parameter $N_{d_{free}}$.

This parameter is defined as the total Hamming weight of all messages associated with the paths at minimum Hamming distance from the all-zero path. We have already identified two paths that are at distance $d_{free} = 4$ from the all-zero path. If we count the total number of ones in the two messages corresponding to both paths, we find $N_{d_{free}} = 3$.

Finally, the bit error probability at high SNR is approximated by

$$P_{eb} \sim \frac{3}{2} \cdot \text{erfc} \left(\sqrt{2 \frac{E_b}{N_0}} \right),$$

and the asymptotic coding gain over uncoded BPSK is equal to $10 \log_{10} 2 \sim 3$ dB.

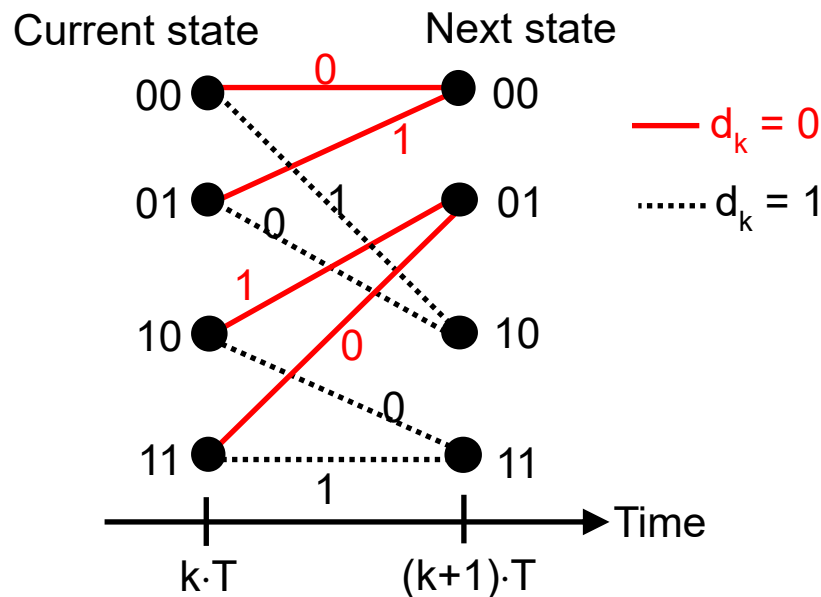
(d) No, the convolutional code studied in this question is not optimal in terms of error performance. In fact, we have seen in class that there are other codes with the same coding rate of $1/2$ and the same constraint length of 3 that can achieve better error performance.

For instance the convolutional code with generator polynomials 5 and 7 have a free distance equal to 5 and is thus able to achieve an asymptotic coding gain over uncoded BPSK of almost 4 dB. This is roughly 1 dB higher than the gain obtained using the code studied here.

(e) As the turbo encoder is systematic, the low-weight codewords can only be generated by encoding low-weight messages.

So, let us consider first weight-1 messages. The trellis determined earlier indicates that a weight-1 message generates a weight-4 component ($M + P_1$) of the codeword.

It also generates a weight-3 component P_2 of the codeword, according to the trellis of the second convolutional code shown below.



Therefore, a weight-1 message generates a weight-7 codeword.

Let us now investigate the effect of weight-2 messages:

If both 1s are successive, then the weight of ($M + P_1$) is $d_1 = 4$ and the weight of P_2 is $d_2 = 2 \rightarrow d_1 + d_2 = 6$.

If both 1s are separated by a 0, then $d_1 = 6$ and $d_2 = 4 \rightarrow d_1 + d_2 = 10$.

If both 1s are separated by two 0s or more, then $d_1 = 8$. and $d_2 = 6 \rightarrow d_1 + d_2 = 14$.

This shows that a weight-2 message generates a codeword whose weight can be as low as 6, which happens only when both 1s are successive.

By investigating the effect of weight-3 messages, we obtain the following results:

If the 1s are successive, then the weight of $(M + P_1)$ is $d_1 = 6$ and the weight of P_2 is $d_2 = 3$.

If the first two 1s are successive and the third one is separated from the second one by a 0, then $d_1 = 6$ and $d_2 = 3$.

And so on. By considering all possibilities, we will find that a weight-3 message does not generate any codeword with a weight of 6 or lower.

We conclude that the minimum Hamming distance of the turbo code is $d_{min} = 6$ and is associated with a weight-2 message in which both 1s are successive.

By using the results given in the lecture notes regarding the effect that a k -bit random interleaver has on weight-2 sequences with successive 1s, we can write $e(d_{min}) = \frac{2}{k}$ and thus

$$P_{eb} \approx \frac{2}{k} \cdot \text{erfc} \left(\sqrt{\frac{6 E_b}{3 N_0}} \right).$$

- (f) Let us compare the first term of the union bound for the turbo encoder studied in (e) with that of the turbo encoder considered in the lecture notes.

We notice that the error coefficient $e(d_{min})$ is identical in both cases, thus indicating a similar beneficial effect of the random interleaver. However, we obtain a lower value for d_{min} (6 instead of 8), which results in a higher error floor.

Due to this lower d_{min} , we would NOT recommend the use of non-recursive convolutional codes for the design of turbo codes. Using recursive and systematic convolutional (RSC) codes provides optimal performance in terms of both interleaver gain and minimum distance between codewords.

We should also mention the detrimental effect that weight-1 messages have on the interleaver gain because it is obvious that an interleaver cannot break a weight-1 message. No interleaver gain could be obtained on a weight-1 message.

For the turbo encoder previously considered, a weight-1 message generates a weight-7 codeword. It would be easy to show that the error coefficient associated with the distance $d = 7$ (corresponding to the second term of the union bound) is in fact very large and independent from the interleaver size.

Remember that weight-1 messages cannot exist for turbo codes designed using RSC encoders because the termination of such encoders in the zero state requires messages to have at least two 1s.

Finally, we can also note that the spectral thinning effect due to the random interleaver is rather limited with non-recursive codes. For instance, if we consider the weight-2 messages, we can observe that the weight of the corresponding codewords can only be equal to 6, 10, or 14.

Question 2

- (a) We can determine all possible codewords by applying the equation

$$C = M \cdot G = (m_0 \ m_1 \ m_2) \cdot \begin{bmatrix} 100110 \\ 010101 \\ 001011 \end{bmatrix}.$$

The six coded bits can thus be obtained using the following set of equations:

$$\begin{aligned} c_0 &= m_0; \\ c_1 &= m_1; \\ c_2 &= m_2; \\ c_3 &= m_0 + m_1; \\ c_4 &= m_0 + m_2; \\ c_5 &= m_1 + m_2. \end{aligned}$$

The encoding table is thus given by

Message M	Codeword C
000	000000
001	001011
010	010101
011	011110
100	100110
101	101101
110	110011
111	111000

To assess the minimum distance d_{min} between codewords, we search in this table for the non-zero codeword(s) with the smallest Hamming weight. We find $d_{min} = 3$.

(b) We need to compute the value of the error coefficient $e(d_{min}) = \frac{W_{d_{min}}}{2k}$ using the encoding table. We find $W_{d_{min}} = 6$.

Therefore, we can finally write $P_{eb} \sim \text{erfc} \left(\sqrt{\frac{3}{2} \frac{E_b}{N_0}} \right)$.

The asymptotic coding gain over uncoded BPSK is equal to $10 \log_{10} \left(\frac{3}{2} \right) \sim 1.76 \text{ dB}$.

(c) We can start from the set of equations that express the six coded bits as a function of the message bits:

$$\begin{aligned}
 c_0 &= m_0; \\
 c_1 &= m_1; \\
 c_2 &= m_2; \\
 c_3 &= m_0 + m_1; \\
 c_4 &= m_0 + m_2; \\
 c_5 &= m_1 + m_2.
 \end{aligned}$$

This set can be rewritten as

$$\begin{aligned}
 c_3 &= c_0 + c_1; \\
 c_4 &= c_0 + c_2;
 \end{aligned}$$

$$c_5 = c_1 + c_2.$$

This is equivalent to

$$c_0 + c_1 + c_3 = 0;$$

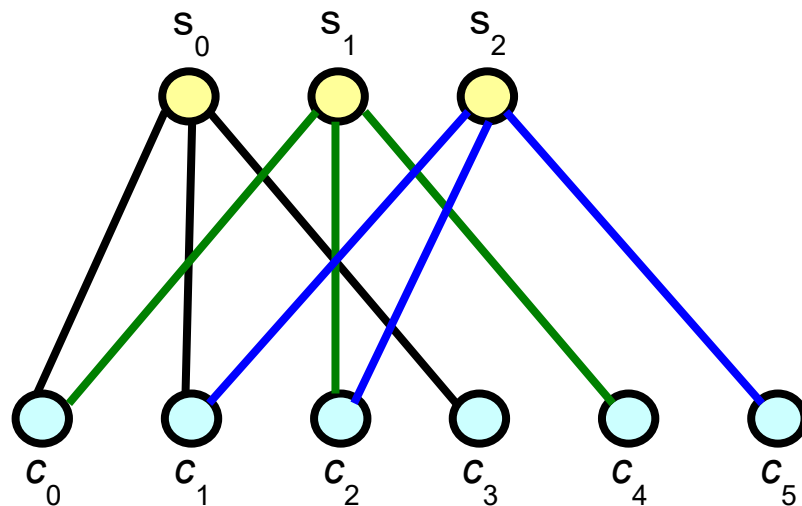
$$c_0 + c_2 + c_4 = 0;$$

$$c_1 + c_2 + c_5 = 0.$$

This set of three equations can be written in matrix form as

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 \end{bmatrix}.$$

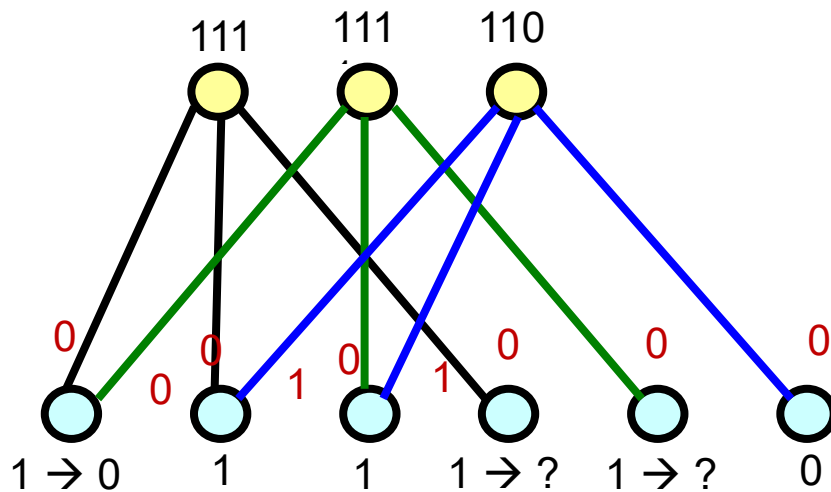
(d)



(e) Assume that the BSC output is $R = (111110)$ and let us see what the result of the bit-flipping algorithm is going to be.

This bit-flipping decoder does not work well because it is not able to recover the codeword (011110) , which would be the result if we used ML decoding.

The reason is that the last three bit nodes are not connected to enough check nodes, which implies that the parity-check matrix/Tanner graph is poorly designed.



(f) We can manipulate the three parity-check equations shown below so as to come up with an improved Tanner graph: .

$$c_0 + c_1 + c_3 = 0;$$

$$c_0 + c_2 + c_4 = 0;$$

$$c_1 + c_2 + c_5 = 0.$$

1st + 2nd equation leads to $c_1 + c_2 + c_3 + c_4 = 0$.

1st + 3rd equation leads to $c_0 + c_2 + c_3 + c_5 = 0$.

2nd + 3rd equation leads to $c_0 + c_1 + c_4 + c_5 = 0$.

1st + 2nd + 3rd equation leads to $c_3 + c_4 + c_5 = 0$.

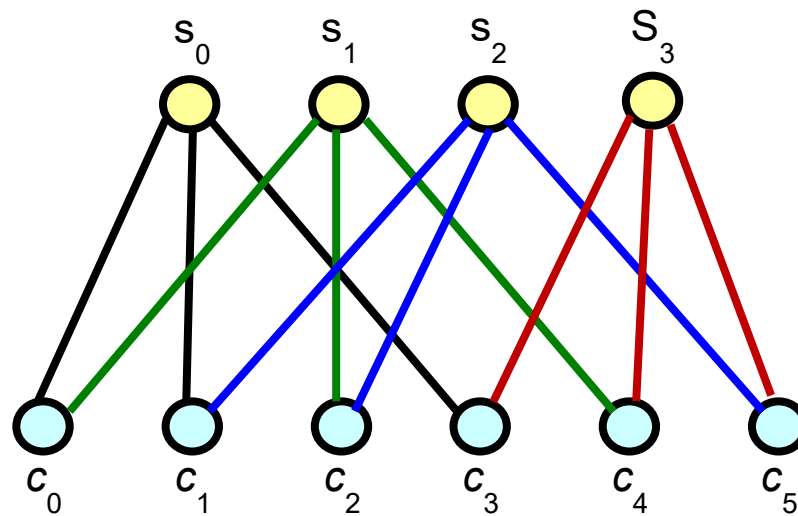
To design our new parity-check matrix/Tanner graph, we can choose to use any of these dependent equations. To minimise the complexity of the decoding, we choose to add only the last equation to the three original ones:

$$c_0 + c_1 + c_3 = 0;$$

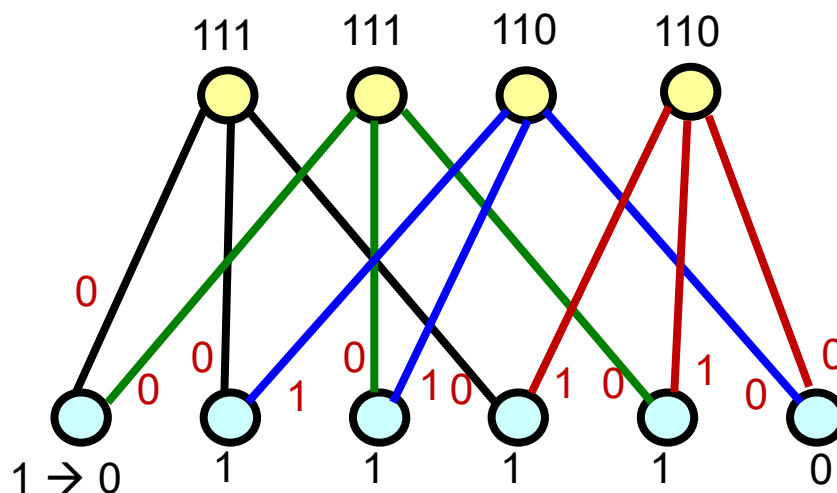
$$c_0 + c_2 + c_4 = 0;$$

$$c_1 + c_2 + c_5 = 0;$$

$$c_3 + c_4 + c_5 = 0.$$



Now, we can see that this Tanner graph is able to decode any received binary word with a single transmission error. For instance, we can consider the same example as above: $R = (111110)$.



This bit-flipping decoder does work very well now because it recovers the codeword (011110) , which would be the result if we used ML decoding.

- END -